

UNIVERSITÉ DU SINE SALOUM EL HADJI IBRAHIMA NIASS

UFR des Sciences Fondamentales et de l'Ingénierie
Département de Mathématiques-Informatique

Travaux Dirigés Programmation Arduino

Thème : Internet et Réseaux (Ethernet)

Enseignant : Dr B. DIOP

Objectif : Maîtriser la connectivité réseau de l'Arduino via le Shield Ethernet, en intégrant des techniques de parsing de données et d'optimisation mémoire.

Partie I : Configuration Réseau et Serveur Web

Exercice 1. Configuration réseau avec fallback

On souhaite configurer le Shield Ethernet de manière robuste : d'abord tenter une connexion DHCP, puis basculer sur une adresse IP statique en cas d'échec.

Travail demandé :

1. Écrire une fonction `initialiserReseau()` qui :
 - Tente d'obtenir une IP via DHCP (timeout de 10 secondes).
 - En cas d'échec, configure une IP statique de secours.
 - Retourne `true` si la connexion a réussi, `false` sinon.
2. Afficher sur le port série le mode utilisé (DHCP ou Statique) et l'adresse IP obtenue.
3. Stocker l'adresse IP de secours en EEPROM pour qu'elle soit modifiable sans recompilation.

```
1 #include <SPI.h>
2 #include <Ethernet.h>
3 #include <EEPROM.h>
4
5 byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };
6
7 // IP de secours stockee en EEPROM (adresses 0-3)
8 IPAddress ipSecours;
9
10 bool initialiserReseau() {
11     // A COMPLETER
12 }
13
14 void setup() {
15     Serial.begin(9600);
```

```

16   if (initialiserReseau()) {
17       Serial.print(F("Connecte avec IP : "));
18       Serial.println(Ethernet.localIP());
19   } else {
20       Serial.println(F("Echec de connexion reseau !"));
21   }
22 }

```

Question : Pourquoi utilise-t-on `F()` pour les chaînes de caractères dans `Serial.print()` ?

Exercice 2. Serveur Web multi-capteurs

Créer un serveur Web qui affiche les valeurs de plusieurs capteurs sous forme de tableau HTML, avec rafraîchissement automatique.

Spécifications :

- Lire 4 entrées analogiques (A0 à A3) représentant différents capteurs.
- Afficher les valeurs dans un tableau HTML avec les colonnes : Capteur, Valeur brute, Pourcentage.
- Ajouter une balise `<meta http-equiv="refresh" content="5">` pour rafraîchir la page toutes les 5 secondes.
- Utiliser `F()` pour toutes les chaînes HTML statiques.

Travail demandé :

1. Écrire la fonction `envoyerPageHTML(EthernetClient& client)` qui génère la page.
2. Calculer la consommation RAM théorique si on n'utilisait pas `F()`.
3. Stocker les noms des capteurs dans un tableau en `PROGMEM`.

```

1  // Noms des capteurs en Flash
2  const char capteur0[] PROGMEM = "Temperature";
3  const char capteur1[] PROGMEM = "Humidite";
4  const char capteur2[] PROGMEM = "Luminosite";
5  const char capteur3[] PROGMEM = "Pression";
6
7  const char* const nomsCapteurs[] PROGMEM = {
8      capteur0, capteur1, capteur2, capteur3
9  };
10
11 void envoyerPageHTML(EthernetClient& client) {
12     // A COMPLETER
13 }

```

Exercice 3. Interface de contrôle domotique

Créer une interface Web permettant de contrôler 4 relais (LEDs pour la simulation) et d'afficher leur état actuel.

Spécifications :

- Chaque relais peut être allumé ou éteint via un lien cliquable.
- L'URL contient la commande : `/?relay=2&action=on` ou `/?relay=2&action=off`.

- La page affiche l'état actuel de chaque relais avec une couleur (vert = ON, rouge = OFF).
- Ajouter un bouton "Tout allumer" et "Tout éteindre".

Travail demandé :

1. Écrire une fonction `parserRequete(String requete)` qui extrait le numéro de relais et l'action demandée.
2. Stocker l'état des 4 relais dans un seul octet (1 bit par relais) pour économiser la RAM.
3. Utiliser les opérations bit à bit pour manipuler les états.

```
1 byte etatRelais = 0b00000000; // 4 bits utilises (relais 0-3)
2
3 void setRelais(int numero, bool etat) {
4     if (etat) {
5         etatRelais |= (1 << numero); // Mettre a 1
6     } else {
7         etatRelais &= ~(1 << numero); // Mettre a 0
8     }
9     digitalWrite(pinRelais[numero], etat ? HIGH : LOW);
10 }
11
12 bool getRelais(int numero) {
13     return (etatRelais >> numero) & 1;
14 }
15
16 void parserRequete(String requete) {
17     // Chercher "relay=" et "action=" dans la requete
18     // A COMPLETER
19 }
```

Question algorithmique : Comment optimiser le parsing de la requête pour minimiser l'utilisation de String (qui fragmente la mémoire) ?

Partie II : Client Web et Parsing de Données

Exercice 4. Client météo avec parsing JSON

L'Arduino interroge une API météo et extrait la température actuelle depuis une réponse JSON.

Exemple de réponse JSON simplifiée :

```
{"city":"Dakar","temp":28.5,"humidity":65,"wind":12.3}
```

Travail demandé :

1. Écrire une fonction `extraireValeur(char* json, char* cle)` qui recherche une clé et retourne sa valeur numérique.
2. Implémenter un parsing "à la volée" qui lit le flux caractère par caractère sans stocker toute la réponse.
3. Gérer les cas d'erreur (clé non trouvée, valeur invalide).

```

1 // Machine a etats pour le parsing
2 enum EtatParsing { CHERCHE_CLE, LIT_CLE, CHERCHE_VALEUR, LIT_VALEUR };
3
4 float extraireValeurStream(EthernetClient& client, const char*
   cleCherchee) {
5     EtatParsing etat = CHERCHE_CLE;
6     char buffer[16]; // Buffer minimal pour la cle/valeur
7     int index = 0;
8
9     while (client.connected() && client.available()) {
10         char c = client.read();
11
12         switch (etat) {
13             case CHERCHE_CLE:
14                 // A COMPLETER : detecter le debut d'une cle (guillemet)
15                 break;
16             case LIT_CLE:
17                 // A COMPLETER : accumuler les caracteres de la cle
18                 break;
19             // ...
20         }
21     }
22     return NAN; // Cle non trouvee
23 }

```

Contrainte mémoire : Le buffer ne doit pas dépasser 32 octets au total.

Exercice 5. Agrégateur de données multi-sources

L'Arduino interroge successivement 3 serveurs différents, collecte une valeur de chacun, puis calcule des statistiques.

Spécifications :

- Tableau de 3 serveurs avec leurs URLs.
- Pour chaque serveur : connexion, requête GET, extraction d'une valeur numérique.
- Calcul de la moyenne, min et max des 3 valeurs récupérées.
- Gestion des timeouts (5 secondes max par serveur).

Travail demandé :

1. Stocker la liste des serveurs en PROGMEM.
2. Écrire une fonction `interrogerServeur(int index)` qui retourne la valeur ou NAN en cas d'échec.
3. Calculer les statistiques uniquement sur les valeurs valides.

```

1 // Serveurs stockes en Flash
2 const char serveur0[] PROGMEM = "api.capteur1.com";
3 const char serveur1[] PROGMEM = "api.capteur2.com";
4 const char serveur2[] PROGMEM = "api.capteur3.com";
5
6 const char* const serveurs[] PROGMEM = { serveur0, serveur1, serveur2 };
7 const char* const chemins[] PROGMEM = { "/valeur", "/data", "/sensor" };
8

```

```

9 float valeurs[3];
10 int nbValeursValides = 0;
11
12 void collecterDonnees() {
13     for (int i = 0; i < 3; i++) {
14         valeurs[i] = interrogerServeur(i);
15         if (!isnan(valeurs[i])) {
16             nbValeursValides++;
17         }
18     }
19 }
20
21 float calculerMoyenne() {
22     // Calculer la moyenne des valeurs non-NAN
23     // A COMPLETER
24 }

```

Exercice 6. Buffer circulaire pour historique réseau

Créer un système qui stocke les 20 dernières valeurs reçues du réseau dans un buffer circulaire, avec horodatage.

Spécifications :

- Chaque entrée : valeur (float) + timestamp (unsigned long) + source (byte).
- Méthodes : `ajouter()`, `getMoyenne()`, `getValeur(int i)` (0 = plus récent).
- Affichage de l'historique sur la page Web du serveur.

Travail demandé :

1. Définir la structure `Enregistrement` (taille minimale).
2. Implémenter le buffer circulaire avec gestion correcte des index.
3. Calculer la mémoire totale utilisée par le buffer.

```

1 struct Enregistrement {
2     float valeur;           // 4 octets
3     unsigned long timestamp; // 4 octets
4     byte source;           // 1 octet
5 }; // Total : 9 octets x 20 = 180 octets
6
7 class HistoriqueReseau {
8 public:
9     void ajouter(float val, byte src);
10    float getMoyenne();
11    float getTendance(); // Regression lineaire simplifiee
12    Enregistrement getEnregistrement(int i); // 0 = plus recent
13    int getTaille();
14
15 private:
16    Enregistrement _buffer[20];
17    byte _tete;
18    byte _taille;
19 };

```

Question : Comment calculer la tendance (pente) des données pour détecter une augmentation ou diminution ?

Partie III : Protocole UDP et Synchronisation

Exercice 7. Communication UDP entre deux Arduino

Deux Arduino sur le même réseau échangent des données de capteurs via UDP (plus rapide que TCP).

Spécifications :

- Arduino A envoie ses lectures de capteurs toutes les 100 ms.
- Arduino B reçoit et affiche les données, détecte les paquets perdus.
- Format du paquet : [ID:2] [SEQ:2] [DATA:4] (8 octets).
- Calcul du taux de perte sur les 100 derniers paquets.

Travail demandé :

1. Écrire le code de l'émetteur (Arduino A).
2. Écrire le code du récepteur (Arduino B) avec détection des pertes.
3. Utiliser un tableau circulaire pour suivre les 100 derniers numéros de séquence.

```
1 // Structure du paquet UDP
2 struct PaquetCapteur {
3     uint16_t idEmetteur;
4     uint16_t numeroSequence;
5     float valeur;
6 };
7
8 // Cote recepteur
9 uint16_t dernierSeq = 0;
10 uint16_t paquetsRecus = 0;
11 uint16_t paquetsPerdus = 0;
12
13 void traiterPaquet(PaquetCapteur* paquet) {
14     uint16_t attendu = dernierSeq + 1;
15     if (paquet->numeroSequence != attendu) {
16         // Calculer le nombre de paquets perdus
17         paquetsPerdus += (paquet->numeroSequence - attendu);
18     }
19     dernierSeq = paquet->numeroSequence;
20     paquetsRecus++;
21 }
22
23 float getTauxPerte() {
24     return (float)paquetsPerdus / (paquetsRecus + paquetsPerdus) *
25     100.0;
26 }
```

Exercice 8. Synchronisation NTP avec gestion du temps

Implémenter une synchronisation de l'heure via le protocole NTP, avec resynchronisation périodique.

Spécifications :

- Interroger un serveur NTP au démarrage et toutes les heures.
- Convertir le timestamp Unix en heure lisible (HH:MM:SS).

- Gérer le fuseau horaire (UTC+0 pour le Sénégal).
- Afficher l'heure sur la page Web du serveur.

Travail demandé :

1. Écrire la fonction `envoyerRequeteNTP()` qui envoie le paquet UDP.
2. Écrire la fonction `lireReponseNTP()` qui extrait le timestamp.
3. Écrire les fonctions de conversion : `getHeures()`, `getMinutes()`, `getSecondes()`.

```
1 #include <EthernetUdp.h>
2
3 EthernetUDP udp;
4 const char serveurNTP[] = "pool.ntp.org";
5 const int portNTP = 123;
6
7 unsigned long epochTime = 0;
8 unsigned long derniereSynchro = 0;
9
10 // Paquet NTP (48 octets)
11 byte paquetNTP[48];
12
13 void envoyerRequeteNTP() {
14     memset(paquetNTP, 0, 48);
15     paquetNTP[0] = 0b11100011; // LI, Version, Mode
16     paquetNTP[1] = 0;          // Stratum
17     paquetNTP[2] = 6;          // Polling Interval
18     paquetNTP[3] = 0xEC;       // Precision
19
20     udp.beginPacket(serveurNTP, portNTP);
21     udp.write(paquetNTP, 48);
22     udp.endPacket();
23 }
24
25 unsigned long lireReponseNTP() {
26     // A COMPLETER : lire les octets 40-43 (timestamp)
27 }
28
29 int getHeures(unsigned long epoch) {
30     return (epoch % 86400L) / 3600;
31 }
```

Partie IV : Projet Intégrateur

Exercice 9. Station météo connectée complète

Créer une station météo qui :

- Lit des capteurs locaux (température, humidité simulés sur A0, A1).
- Récupère des données météo en ligne (API externe).
- Affiche toutes les données sur une page Web avec historique.
- Envoie les données vers un serveur distant (POST HTTP).

Travail demandé :

1. Concevoir l'architecture logicielle (quelles fonctions, quelles structures de données).
2. Implémenter le serveur Web avec affichage des données locales et distantes.
3. Implémenter le client Web pour récupérer et envoyer des données.
4. Utiliser un buffer circulaire pour l'historique (10 dernières mesures).
5. Optimiser la RAM : toutes les chaînes en `PROGMEM`, pas de `String`.

```

1 // Structure principale
2 struct DonneesMeteo {
3     float tempLocale;
4     float humiditeLocale;
5     float tempDistante;
6     unsigned long timestamp;
7 };
8
9 DonneesMeteo historique[10];
10 byte indexHistorique = 0;
11
12 // Fonctions principales
13 void lireCapteurs();
14 void recupererMeteoDistante();
15 void envoyerVersServeur();
16 void servirPageWeb(EthernetClient& client);
17 void mettreAJourHistorique();
18
19 void loop() {
20     // Gestion des clients Web entrants
21     EthernetClient client = server.available();
22     if (client) {
23         servirPageWeb(client);
24     }
25
26     // Mise a jour periodique (toutes les 60 secondes)
27     static unsigned long derniereMaj = 0;
28     if (millis() - derniereMaj > 60000) {
29         lireCapteurs();
30         recupererMeteoDistante();
31         mettreAJourHistorique();
32         envoyerVersServeur();
33         derniereMaj = millis();
34     }
35 }

```

Exercice 10. Système de monitoring multi-nœuds

Concevoir un système où un Arduino “maître” collecte les données de plusieurs Arduino “esclaves” via UDP, puis les affiche sur une interface Web centralisée.

Architecture :

- 1 Arduino Maître : serveur Web + collecteur UDP.
- N Arduino Esclaves : émetteurs UDP périodiques.

- Le maître maintient une table des esclaves avec leurs dernières valeurs et timestamps.

Travail demandé :

1. Définir le protocole de communication (structure des paquets).
2. Sur le maître : maintenir un tableau de 8 esclaves maximum avec détection des nœuds inactifs (timeout 30s).
3. Afficher sur la page Web : liste des esclaves, leurs valeurs, et leur état (actif/inactif).
4. Calculer des statistiques globales (moyenne de tous les esclaves actifs).

```
1 // Structure pour un noeud esclave
2 struct NoeudEsclave {
3     byte id;
4     IPAddress ip;
5     float derniereValeur;
6     unsigned long dernierContact;
7     bool actif;
8 };
9
10 NoeudEsclave esclaves[8];
11 byte nbEsclaves = 0;
12
13 void mettreAJourEsclave(byte id, IPAddress ip, float valeur) {
14     // Chercher l'esclave dans le tableau
15     int index = -1;
16     for (int i = 0; i < nbEsclaves; i++) {
17         if (esclaves[i].id == id) {
18             index = i;
19             break;
20         }
21     }
22
23     // Si nouveau, l'ajouter
24     if (index == -1 && nbEsclaves < 8) {
25         index = nbEsclaves++;
26         esclaves[index].id = id;
27         esclaves[index].ip = ip;
28     }
29
30     // Mettre a jour
31     if (index >= 0) {
32         esclaves[index].derniereValeur = valeur;
33         esclaves[index].dernierContact = millis();
34         esclaves[index].actif = true;
35     }
36 }
37
38 void verifierTimeouts() {
39     unsigned long maintenant = millis();
40     for (int i = 0; i < nbEsclaves; i++) {
41         if (maintenant - esclaves[i].dernierContact > 30000) {
42             esclaves[i].actif = false;
43         }
44     }
45 }
46
```

```

47 float getMoyenneGlobale() {
48     float somme = 0;
49     int nbActifs = 0;
50     for (int i = 0; i < nbEsclaves; i++) {
51         if (esclaves[i].actif) {
52             somme += esclaves[i].derniereValeur;
53             nbActifs++;
54         }
55     }
56     return (nbActifs > 0) ? somme / nbActifs : NAN;
57 }

```

Questions de réflexion :

1. Comment gérer la suppression d'un esclave qui n'émet plus depuis longtemps ?
2. Comment éviter les conflits si deux esclaves ont le même ID ?
3. Quelle serait la limite du nombre d'esclaves en termes de mémoire RAM ?

Questions de Synthèse

Question A : Comparaison TCP vs UDP

Compléter le tableau suivant et donner un exemple d'utilisation appropriée pour chaque protocole dans un contexte Arduino :

Critère	TCP	UDP
Fiabilité	?	?
Vitesse	?	?
Ordre des paquets	?	?
Overhead (surcharge)	?	?
Cas d'usage Arduino	?	?

Question B : Optimisation mémoire pour le Web

Soit le code suivant qui consomme beaucoup de RAM :

```

1 client.println("<html><head><title>Ma Page</title></head>");
2 client.println("<body><h1>Bienvenue</h1>");
3 client.println("<p>Temperature : " + String(temp) + " C</p>");
4 client.println("</body></html>");

```

1. Calculer approximativement la consommation RAM de ce code.
2. Réécrire ce code de manière optimisée (utiliser `F()`, éviter `String`).
3. Proposer une technique pour générer du HTML dynamique sans `String`.

Question C : Sécurité réseau

L'Arduino avec Shield Ethernet ne peut pas facilement gérer le HTTPS.

1. Expliquer pourquoi le chiffrement SSL/TLS est difficile sur un microcontrôleur 8 bits.
2. Proposer des mesures de sécurité alternatives pour protéger une installation Arduino en réseau local.

3. Quels risques encourt-on si l'Arduino est exposé directement sur Internet ?

Bon courage !